SIC 8 · BIG DATA ENGINEERING · PROJECT 03 · PROJECT PROPOSAL

Enterprise Fleet Telematics Lakehouse & Dynamic Logistics Warehouse

Team of 3 · 6 build days · Hybrid delivery: local Docker platform + AWS cloud pathway running the same code (`ENV=local | aws`)

Problem

Fleet managers cannot combine vehicle diagnostics, position and cargo temperature in one place, so route deviations, refrigeration breaches and excessive idling are hard to investigate consistently.

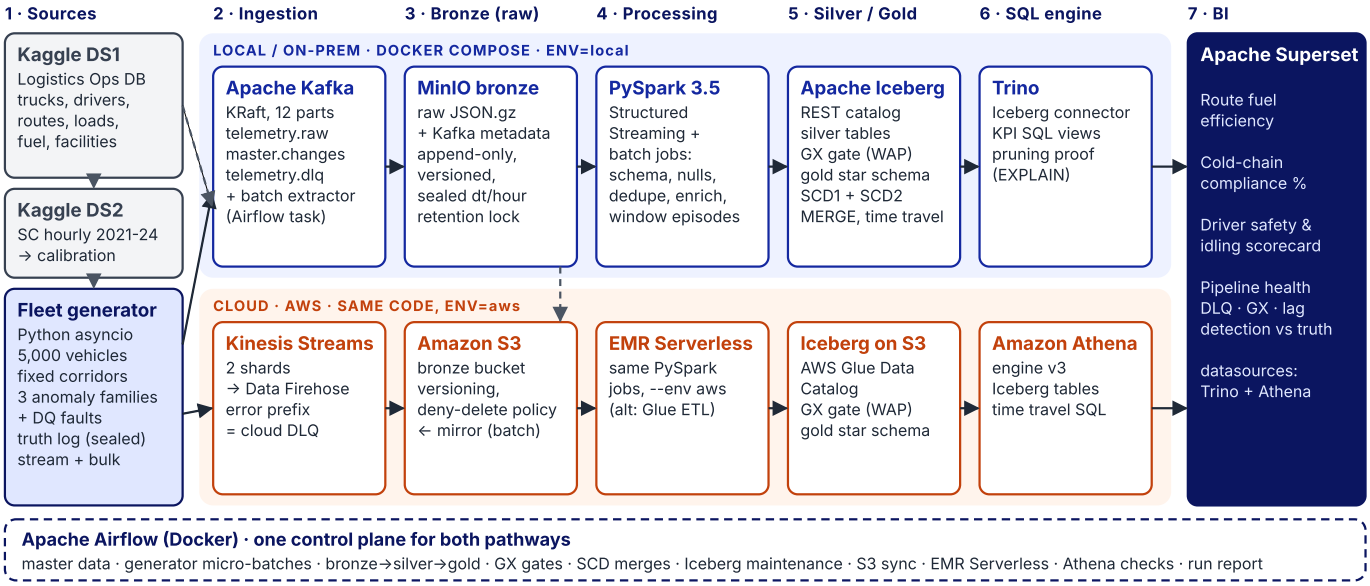
Objective

Build a telematics lakehouse and star-schema warehouse for **5,000 simulated delivery vehicles** on fixed corridors, serving route fuel efficiency, cold-chain compliance % and driver safety/idling scorecards.

Measurable targets

5,000 distinct vehicles · 0 duplicate keys in silver · bronze = silver + quarantine + DLQ · detection recall ≥ 95% / precision ≥ 90% vs injected truth · KPI dashboard = SQL ±0.1% · fresh-clone run ≤ 30 min.

End-to-end flow (local and AWS pathways)



Data work: how the Kaggle data and the generator connect

Source	Role
DS1 Logistics Ops DB (2022–24)	**Master data.** Seeds dim_vehicle, dim_driver, dim_route, dim_facility, dim_cargo; DS1 routes become fixed corridors; DS1 loads become trips; fuel purchases benchmark simulated km/L.
DS2 Supply-chain hourly (2021–24)	**Calibration + history.** Fits hour-of-day speed, fuel-rate, temperature-noise and driver-profile distributions into `calibration.json` ; kept as a historical context fact.
Generator (asyncio + Faker)	Expands DS1 to **5,000 vehicles** (`VH-00001…` keeps `source_truck_id`); emits GPS, OBD-II fuel rate, RPM, engine temp and reefer temp per ping; stream mode (Kafka/Kinesis) + hourly bulk micro-batches; seeded config.

Anomaly	Injected → detected (rules never read labels)
Cold-chain breach	Reefer +4–10 °C for 15–90 min → temp > setpoint + 3 °C for ≥ 15 min
Excessive idling	Engine on, stopped 12–60 min → speed < 2, RPM 500–1000 ≥ 10 min, off-facility
Route deviation	3–15 km off corridor → > 2 km from polyline for ≥ 3 pings
DQ faults	Malformed, nulls, duplicates, late events → DLQ, flags, dedupe, MERGE

Injections go to a sealed truth log the jobs cannot read; a harness scores detections against it. Units are converted to metric at silver; event time is kept separate from wall-clock time.

KPIs (documented definitions)

KPI	Definition
Fuel efficiency / route	Σ km ÷ Σ litres per route (km/L); km = haversine between pings, litres = fuel_rate × Δt
Cold-chain compliance	Reefer minutes within setpoint −2/+3 °C ÷ reefer-active minutes on cold loads (simulation threshold, not certification)
Driver scorecard	Idle % = idle min ÷ engine-on min; safety score = 100 − (5·harsh/100 km + 3·speeding/100 km + 0.5·idle %), 0–100

Processing, lakehouse and serving design

Ingestion & bronze

- Kafka topics `telemetry.raw` (12 partitions by vehicle_id), `master.changes`, `telemetry.dlq`; Kinesis → Firehose in AWS.
- Retry with exponential backoff + jitter, DLQ with error codes, replay tool, `ops.ingestion_log`.
- Append-only raw JSON + Kafka metadata on MinIO / S3, versioned, sealed partitions locked.

PySpark silver

- Contract schema enforcement; null policy with sensor-gap flags; quarantine.
- Dedupe on (vehicle_id, event_ts, seq_no) with watermark; broadcast joins to dims.
- Window functions: distance/fuel deltas, idle / excursion / deviation episodes. Snappy Parquet, partitioned by day + vehicle bucket.

Iceberg gold & quality

- Star schema with surrogate keys: SCD2 dim_vehicle, dim_driver; SCD1 dim_route.
- Facts: telemetry ping, vehicle-hourly, trip, 3 episode facts, driver-daily; additivity documented.
- GX gate on an Iceberg audit branch before publish (WAP); MERGE, time travel, schema evolution, pruning, compaction.

Tools and alternatives

Layer	Local (chosen)	AWS (chosen)	Alternatives
Generator	Python 3.11 asyncio, Faker, numpy	Same code, Kinesis sink	threading/multiprocessing, Mimesis, SDV
Contract	JSON + JSON Schema (v1, v2)	Same	Avro + Schema Registry, Protobuf
Broker	Apache Kafka (KRaft)	Kinesis Data Streams + Firehose	Redpanda, Amazon MSK, NiFi (batch)
Raw lake	MinIO	Amazon S3	HDFS, Ozone
Processing	PySpark 3.5 standalone	EMR Serverless	AWS Glue ETL, Databricks, Dataproc
Table / catalog	Iceberg + REST catalog	Iceberg + Glue Data Catalog	Delta Lake, Hudi; Nessie, Hive Metastore
SQL engine	Trino	Amazon Athena	Presto, Hive on Tez, ClickHouse, Snowflake
Quality	Great Expectations	Same (via Airflow)	Soda Core, PyDeequ, Spark assertions
Orchestration	Airflow (Docker Compose)	Same Airflow drives AWS operators	Amazon MWAA, Dagster, Prefect
BI	Apache Superset	Superset → Athena	Metabase, Power BI, QuickSight
Infra / dev	Docker Compose, Makefile, GitHub, pytest	Terraform, AWS Budgets, CloudWatch	CDK, CloudFormation, GitLab CI

Work distribution

Member A · Data & Streaming

Kaggle profiling, data contract, DS2 calibration, corridors, 5,000-vehicle generator, anomalies + truth log, Kafka/Kinesis producers, retry/DLQ/replay, ingestion log, detection-vs-truth harness, benchmarks.

Member B · Processing & Lakehouse

Bronze stream job, silver cleaning/dedupe/enrichment, star schema, SCD1/SCD2 MERGE, episode detection (independent of A), KPI marts, GX + WAP gates, Iceberg features, tuning.

Member C · Platform, Cloud & Analytics

Repo/CI, Docker Compose, Terraform for AWS + budget alarms, Airflow DAGs for both paths, Trino/Athena, Superset dashboards, KPI reconciliation, README, evidence, demo video, deck lead.

6-day timeline (handbook readiness gates)

Day 1 · Gate 1
Profile data, contract, ERD + grain, KPI formulas, Docker stack, AWS base. *Exit:* scope agreed, stack healthy.

Day 2 · Gate 2
Generator at 5,000; bronze stream; master dims; Kinesis → S3 → Athena. *Exit:* one event traced in both paths.

Day 3 · Core I
Anomalies + DQ faults; full silver; SCD dims; first facts; pipeline DAG skeleton.

Day 4 · Gate 3
Episode facts, scorecards, KPI marts, GX + WAP, dashboards, EMR/Athena run. *Exit:* raw → dashboard.

Day 5 · Gate 4
16 tests: failures, duplicates, SCD, time travel, pruning, KPI reconciliation, fresh-clone run.

Day 6 · Gate 5 prep
Freeze 12:00, final full run, evidence pack, deck, recorded fallback, 2 rehearsals.

Daily 17:00 end-to-end integration on the demo machine; every PR reviewed by a teammate; AWS resources run only in test windows and are torn down with `make aws-down`.